

Báo cáo Nghiên cứu Chuyên sâu Toàn diện: Agent Skills KIT

Mục lục nhanh

- 1. Tổng quan Dự án
- 2. Kiến trúc Hệ thống
- 3. Meta-Skill: `using-agent-skills`
- 4. Hệ thống 20+1 Kỹ năng
- 5. Giải phẫu một Kỹ năng
- 6. Phân tích 7 Lệnh
- 7. Hệ thống Personas
- 8. Orchestration Patterns
- 9. Hooks System
- 10. Tương thích Đa nền tảng
- 10.1. Cài đặt với Cursor
- 10.2. Cài đặt với Gemini CLI
- 10.3. Các nền tảng khác
- 11. Tổng kết Đánh giá
- 12. Demo Full Workflow với PRD TaskFlow
- 12.1. Mục tiêu Demo
- 12.2. Input từ PRD và giả định
- 12.3. Phạm vi 12 FR và các US
- 12.4. Full Workflow theo `agent-skills`
- 12.5. Kịch bản Trình bày cho Toàn Team
- 12.6. Prompt mẫu cho từng bước
- 12.7. Giá trị Rút ra từ Demo
- 12.8. Kết luận cho phần Demo

1. Tổng quan Dự án (Project Overview)

1.1. Khái niệm và Mục đích cốt lõi

Agent Skills là một bộ công cụ cung cấp các "kỹ năng kỹ thuật chuẩn sản xuất" (production-grade engineering skills) dành riêng cho các AI Coding Agents (Claude Code, Cursor, Copilot, OpenCode, Windsurf, Gemini CLI...). Vấn đề lớn nhất của các AI Agent hiện nay là **xu hướng chọn "con đường ngắn nhất"** – bỏ qua viết tài liệu, kiểm thử, đánh giá bảo mật. Agent Skills giải quyết triệt để bằng cách ép AI tuân thủ workflows, quality gates và best practices từ văn hóa kỹ sư Google (TDD, Chesterton's Fence, trunk-based development). (Nguồn: `README.md`, dòng 1-5, 278-282)

1.2. Triết lý Thiết kế (Design Principles)

4 nguyên tắc cốt lõi:

- **Process, not prose:** Kỹ năng là workflows AI bắt buộc phải theo, không phải văn xuôi để "tham khảo". Mỗi kỹ năng có bước, điểm kiểm tra và tiêu chí thoát rõ ràng.
- **Anti-rationalization:** Mọi kỹ năng chứa bằng chứng ngăn chặn AI hay dùng (như "tôi sẽ thêm test sau") kèm lý lẽ phản bác.
- **Verification is non-negotiable:** Luôn yêu cầu bằng chứng thực tế (test passing, build output, runtime data). "Có vẻ đúng" là không đủ.
- **Progressive disclosure:** Chỉ load thông tin tổng quan lúc đầu, references load khi thực sự cần, tiết kiệm token. (Nguồn: `README.md`, dòng 231-237)

1.3. Sáu Hành vi Bắt buộc Xuyên suốt (Core Operating Behaviors)

Đây là 6 quy tắc **phi thương lượng** áp dụng cho mọi skill:

1. **Surface Assumptions** — Nêu rõ giả định trước khi code. Không âm thầm tự quyết.
2. **Manage Confusion Actively** — DỪNG NGAY khi gặp mâu thuẫn. Nêu cụ thể điểm rối. Chờ giải quyết.
3. **Push Back When Warranted** — AI phải phản đối nếu cách tiếp cận sai. "Sycophancy is a failure mode."
4. **Enforce Simplicity** — Nếu build 1000 dòng mà 100 đã đủ, đó là thất bại. Ưu tiên giải pháp đơn giản.
5. **Maintain Scope Discipline** — Chỉ chạm vào phần được yêu cầu. Không "dọn dẹp" code không liên quan.
6. **Verify, Don't Assume** — Mọi task chưa hoàn thành cho đến khi có bằng chứng xác minh. (Nguồn: `skills/using-agent-skills/SKILL.md`, dòng 39-107)

2. Kiến trúc Hệ thống (Architecture)

2.1. Cấu trúc Phân tầng (3 Orchestration Layers)

Tầng	Đường dẫn	Vai trò	Ví dụ
Skills (The How)	<code>skills/<name>/SKILL.md</code>	Quy trình bắt buộc khi intent khớp	<code>code-review-and-quality</code>
Personas (The Who)	<code>agents/<role>.md</code>	Vai trò + góc nhìn + định dạng output	<code>code-reviewer</code>
Commands (The When)	<code>.claude/commands/*.md</code>	Điều phối, gọi skills/personas	<code>/review</code> , <code>/ship</code>

Quy tắc vàng: User (hoặc slash command) là orchestrator. **Personas KHÔNG ĐƯỢC gọi personas khác.** Skills là hops bắt buộc bên trong persona. (Nguồn: `AGENTS.md`, dòng 68-80; `agents/README.md`, dòng 21)

2.2. Cấu trúc Thư mục

- `skills/` : **21 thư mục** — 20 kỹ năng kỹ thuật + 1 meta-skill (`using-agent-skills`).
- `agents/` : 3 persona chuyên gia + README hướng dẫn.
- `references/` : **5 file** tài liệu bổ trợ (Testing, Security, Performance, Accessibility, **Orchestration Patterns**).
- `hooks/` : Hệ thống hook vòng đời session (Session Start, SDD-Cache, Simplify-Ignore).
- `.claude/commands/` và `.gemini/commands/` : 7 slash commands cho mỗi nền tảng.
- `docs/` : Tập tài liệu hỗn hợp, gồm setup guides cho một số nền tảng chính và tài liệu dùng chung như `getting-started.md`, `skill-anatomy.md`.

Cây cấu trúc rút gọn của bộ KIT:

```

agent-skills/
├── README.md           # Giới thiệu tổng quan, quick start, danh sách skills/c
├── AGENTS.md           # Luật orchestration và cách map intent → skill
├── CLAUDE.md           # Hướng dẫn riêng khi dùng với Claude/Claude Code
├── CONTRIBUTING.md     # Quy tắc đóng góp, viết thêm skill mới
├── LICENSE             # Giấy phép sử dụng
├── .claude/
│   └── commands/       # 7 slash commands cho Claude Code: /spec, /plan, /buil
├── .gemini/
│   └── commands/       # Bộ command tương đương cho Gemini CLI dưới dạng .toml
├── .opencode/
│   └── skills/         # Điểm tích hợp để OpenCode load skills
├── .github/
│   └── workflows/     # CI kiểm tra plugin/cài đặt và workflow liên quan

```

```

├─ .claude-plugin/          # Metadata/plugin packaging cho Claude Code marketplace
├─ skills/
│   ├─ using-agent-skills/  # Meta-skill: chọn đúng skill theo loại task
│   ├─ spec-driven-development/ # Skill viết spec trước khi code
│   ├─ planning-and-task-breakdown/ # Skill chia việc thành vertical slices
│   ├─ incremental-implementation/ # Skill build từng lát nhỏ, có verify
│   └─ ...                  # Các skill còn lại cho test, review, ship, security ...
├─ agents/
│   ├─ code-reviewer.md     # Persona review code theo 5 trục
│   ├─ security-auditor.md  # Persona audit security
│   ├─ test-engineer.md     # Persona đánh giá test strategy/coverage
│   └─ README.md            # Giải thích vai trò và quy tắc composition của persona
├─ references/
│   ├─ testing-patterns.md  # Checklist/pattern cho test
│   ├─ security-checklist.md # Checklist bảo mật
│   ├─ performance-checklist.md # Checklist hiệu năng
│   └─ orchestration-patterns.md # Mẫu orchestration đúng/sai
├─ hooks/
│   ├─ hooks.json           # Khai báo hook được plugin dùng
│   ├─ session-start.sh     # Inject meta-skill vào session mới
│   ├─ sdd-cache-pre.sh     # Hook cache cho source-driven-development
│   ├─ sdd-cache-post.sh    # Hook ghi/revalidate cache sau khi fetch docs
│   └─ simplify-ignore.sh   # Hook bảo vệ vùng code không được simplify
└─ docs/
    ├─ getting-started.md   # Cách dùng chung cho agent bất kỳ
    ├─ skill-anatomy.md     # Chuẩn cấu trúc của một SKILL.md
    ├─ cursor-setup.md      # Hướng dẫn tích hợp với Cursor
    ├─ gemini-cli-setup.md  # Hướng dẫn tích hợp với Gemini CLI
    └─ ...                  # Các tài liệu setup/hướng dẫn còn lại

```

(Nguồn: `README.md`, mục *Quick Start*; thư mục `references/`, `hooks/`, `docs/`)

3. Meta-Skill: `using-agent-skills` — Bộ não Điều phối

3.1. Skill Discovery Flowchart

Khi nhận task, AI phải dùng cây quyết định sau để chọn skill:

Task arrives

— Ý tưởng mơ hồ?	→ idea-refine
— Dự án/tính năng mới?	→ spec-driven-development
— Có spec, cần chia task?	→ planning-and-task-breakdown
— Đang code?	→ incremental-implementation
— UI?	→ frontend-ui-engineering
— API?	→ api-and-interface-design
— Cần context tốt hơn?	→ context-engineering
— Cần docs chính thức?	→ source-driven-development
— Viết/chạy test?	→ test-driven-development
— Browser-based?	→ browser-testing-with-devtools
— Có lỗi?	→ debugging-and-error-recovery
— Review code?	→ code-review-and-quality
— Bảo mật?	→ security-and-hardening
— Hiệu năng?	→ performance-optimization
— Commit/branch?	→ git-workflow-and-versioning
— CI/CD?	→ ci-cd-and-automation
— Viết docs/ADR?	→ documentation-and-adrs
— Deploy/launch?	→ shipping-and-launch

(Nguồn: `skills/using-agent-skills/SKILL.md`, dòng 16-37)

3.2. Lifecycle Sequence hoàn chỉnh

1. idea-refine	→ Refine ý tưởng
2. spec-driven-development	→ Định nghĩa xây gì
3. planning-and-task-breakdown	→ Chia nhỏ thành chunks
4. context-engineering	→ Load context đúng lúc
5. source-driven-development	→ Đối chiếu docs chính thức
6. incremental-implementation	→ Build từng slice
7. test-driven-development	→ Chứng minh từng slice hoạt động
8. code-review-and-quality	→ Review trước merge
9. git-workflow-and-versioning	→ Commit history sạch
10. documentation-and-adrs	→ Ghi lại quyết định
11. shipping-and-launch	→ Deploy an toàn

(Nguồn: `skills/using-agent-skills/SKILL.md`, dòng 133-149)

4. Hệ thống 20+ 1 Kỹ năng (The Skills)

4.1. Giai đoạn Define (Xác định)

- `idea-refine` : Phân tích hội tụ/phân kỳ để biến ý tưởng mơ hồ thành đề xuất cụ thể.
- `spec-driven-development` : Viết PRD gồm mục tiêu, commands, cấu trúc, code style, testing strategy, boundaries. **Trước khi** viết code. (Nguồn: `README.md`, dòng 134-137)

4.2. Giai đoạn Plan (Kế hoạch)

- `planning-and-task-breakdown` : Chẻ nhỏ Spec thành task có acceptance criteria và dependency ordering. (Nguồn: `README.md`, dòng 141-143)

4.3. Giai đoạn Build (Xây dựng)

- `incremental-implementation` : Cắt lát dọc (Thin vertical slices) — code, test, verify, commit từng phần. Feature flags, safe defaults.
- `context-engineering` : Đóng gói thông tin đúng lúc cho AI — rules files, context packing, MCP integrations.
- `source-driven-development` : Ép mọi quyết định framework phải dựa trên docs chính thức (verified citations).
- `frontend-ui-engineering` : Kiến trúc Component, design systems, state management, WCAG 2.1 AA accessibility.
- `api-and-interface-design` : Contract-first design, Hyrum's Law, One-Version Rule, error semantics. (Nguồn: `README.md`, dòng 147-154)

4.4. Giai đoạn Verify (Kiểm chứng)

- `test-driven-development` : Red-Green-Refactor, kim tự tháp Test (80% unit / 15% integration / 5% E2E), quy tắc Beyonce, DAMP over DRY.
- `browser-testing-with-devtools` : Chrome DevTools MCP — DOM, console logs, network traces, performance profiling.
- `debugging-and-error-recovery` : Triage 5 bước: Reproduce → Localize → Reduce → Fix → Guard. (Nguồn: `README.md`, dòng 147-161; `skills/using-agent-skills/SKILL.md`, dòng 165-167)

4.5. Giai đoạn Review (Đánh giá)

- `code-review-and-quality` : Review 5 trục, severity labels (Nit/Optional/FYI), kích thước lý tưởng (~100 dòng).
- `code-simplification` : Nguyên tắc Chesterton's Fence, Rule of 500, rút gọn không đổi hành vi.
- `security-and-hardening` : OWASP Top 10, auth patterns, secrets management, three-tier boundary.

- `performance-optimization` : Measure-first, Core Web Vitals targets, bundle analysis. (Nguồn: `README.md`, dòng 163-170)

4.6. Giai đoạn Ship (Triển khai)

- `git-workflow-and-versioning` : Trunk-based dev, atomic commits, commit-as-save-point pattern.
- `ci-cd-and-automation` : Shift Left, Faster is Safer, feature flags, quality gate pipelines.
- `deprecation-and-migration` : Code-as-liability, compulsory vs advisory deprecation, zombie code removal.
- `documentation-and-adrs` : Architecture Decision Records, inline docs — document the *why*.
- `shipping-and-launch` : Pre-launch checklists, staged rollouts, rollback procedures, monitoring.
(Nguồn: `README.md`, dòng 174-180)

4.7. Meta-Skill

- `using-agent-skills` : Kỹ năng điều phối — chứa Skill Discovery Flowchart, 6 Core Behaviors, 10 Failure Modes. Được inject tự động vào mọi session qua hook `session-start.sh`. (Nguồn: `skills/using-agent-skills/SKILL.md`, dòng 1-175; `hooks/session-start.sh`)

5. Giải phẫu một Kỹ năng (Skill Anatomy)

Mọi `SKILL.md` tuân thủ chuẩn thống nhất:

1. **Frontmatter (YAML)**: `name` (kebab-case, khớp tên thư mục) + `description` (tối đa 1024 ký tự, bao gồm "Use when..." trigger).
2. **Overview**: Elevator pitch — skill này làm gì, tại sao cần theo.
3. **When to Use**: Trigger conditions + exclusions (khi NÀO không dùng).
4. **Core Process**: Workflow bước-bước. **Specific over general** — "Run `npm test`" thay vì "make sure tests work".
5. **Common Rationalizations**: Bảng nguy biện AI + cách phản bác.
6. **Red Flags**: Dấu hiệu tiến trình đang sai hướng.
7. **Verification**: Checklist exit criteria — mỗi checkbox phải có bằng chứng.

6 Writing Principles:

- Process over knowledge | Specific over general | Evidence over assumption
- Anti-rationalization | Progressive disclosure | **Token-conscious** (section nào không ảnh hưởng hành vi AI → xóa)

Quy tắc file hỗ trợ: Chỉ tạo khi vượt 100 dòng. References đặt ở `references/`, KHÔNG trong thư mục `skill`. (Nguồn: `docs/skill-anatomy.md`, dòng 1-129; `CONTRIBUTING.md`, dòng 12-43)

6. Phân tích 7 Lệnh (Commands Deep Dive)

6.1. `/spec` — Xác định Yêu cầu

- AI hỏi user về 4 khía cạnh: Objective, Core features, Tech stack, Boundaries.
- Tạo `SPEC.md` gồm 6 phần cấu trúc.
- Dừng chờ User phê duyệt** mới tiếp tục. (Nguồn: `.claude/commands/spec.md`, dòng 5-15)

6.2. `/plan` — Chia nhỏ Kế hoạch

- AI ở chế độ **read-only** — không được sửa code.
- Vẽ Dependency Graph giữa components.
- Cắt theo **Vertical Slices** (một tính năng hoàn chỉnh end-to-end).
- Xuất `tasks/plan.md` và `tasks/todo.md`. (Nguồn: `.claude/commands/plan.md`, dòng 5-16)

6.3. `/build` — Xây dựng Tĩnh tiến

- Nhặt task từ todo. Đọc acceptance criteria.
- RED:** Viết test thất bại. **GREEN:** Code tối thiểu cho pass.
- Chạy regression test + build.
- Commit, đánh dấu hoàn thành, chuyển task tiếp.
- Nếu fail → tự động gọi `debugging-and-error-recovery`. (Nguồn: `.claude/commands/build.md`, dòng 5-18)

6.4. `/test` — Kiểm chứng

- Tính năng mới:** RED → GREEN → Refactor.
- Bug fix (Prove-It pattern):** (1) Viết test tái hiện bug (FAIL) → (2) Xác nhận fail → (3) Fix → (4) Xác nhận PASS → (5) Chạy full suite.
- Browser: tích hợp thêm Chrome DevTools MCP. (Nguồn: `.claude/commands/test.md`, dòng 5-19)

6.5. `/review` — Đánh giá 5 Trục

- Correctness** — Đúng spec? Edge cases? Tests đủ?
- Readability** — Tên biến chuẩn? Logic rõ ràng?
- Architecture** — Đúng pattern? Ranh giới sạch?

- 4. **Security** — Input validated? Secrets safe?
- 5. **Performance** — N+1 queries? Unbounded ops?

Phân loại: **Critical** (phải sửa) | **Important** (nên sửa) | **Suggestion** (cân nhắc). Kèm `file:line` cụ thể.
(Nguồn: `.claude/commands/review.md`, dòng 5-16)

6.6. `/code-simplify` — Đơn giản hóa

- 1. Đọc project conventions, hiểu mục đích code, callers, edge cases.
- 2. Nhận diện: Deep nesting → Guard clauses; Hàm dài → Split; Duplicate → Shared; Dead code → Remove.
- 3. Áp dụng **từng bước một** — chạy test sau mỗi thay đổi.
- 4. Test FAIL → **Revert ngay lập tức**. (Nguồn: `.claude/commands/code-simplify.md`, dòng 5-22)

6.7. `/ship` — Fan-out Orchestrator

- 1. **Phase A (Parallel fan-out)**: Mở **song song** 3 sub-agents: `code-reviewer`, `security-auditor`, `test-engineer`. Chúng đánh giá độc lập, không giao tiếp với nhau.
- 2. **Phase B (Merge)**: AI chính tổng hợp 3 báo cáo + kiểm tra thêm Accessibility, Infrastructure, Documentation.
- 3. **Phase C (Decision)**: Quyết định **GO** hoặc **NO-GO**. Nếu GO, bắt buộc kèm **Rollback Plan** (trigger conditions, procedure, recovery time objective).

Ngoại lệ: Bỏ qua fan-out nếu **tất cả** điều kiện sau đúng: ≤2 files, <50 dòng diff, không chạm `auth/payments/data/config`. (Nguồn: `.claude/commands/ship.md`, dòng 5-73)

7. Hệ thống Personas (Specialist Agents)

7.1. Ba Persona Chuyên gia

Persona	Vai trò	Góc nhìn
<code>code-reviewer</code>	Senior Staff Engineer	Review 5 trục, verdict APPROVE/REQUEST CHANGES
<code>security-auditor</code>	Security Engineer	OWASP, threat modeling, CVE, auth/authz
<code>test-engineer</code>	QA Specialist	Test strategy, coverage analysis, Prove-It pattern

Persona	Vai trò	Góc nhìn
(Nguồn: <code>README.md</code> , dòng 188-192; <code>agents/code-reviewer.md</code>)		

7.2. Composition Rules (Quy tắc Phối hợp)

Mỗi persona kết thúc bằng block `## Composition` quy định rõ:

- **Gọi trực tiếp khi:** User yêu cầu review/audit/test cụ thể.
- **Gọi qua command:** `/review`, `/ship`.
- **KHÔNG BAO GIỜ gọi từ persona khác.** Nếu `code-reviewer` thấy cần security audit → chỉ **recommend** trong báo cáo, không tự gọi `security-auditor`.

Ràng buộc này được Claude Code **cưỡng chế ở cấp nền tảng**: subagents cannot spawn subagents.
(Nguồn: `agents/code-reviewer.md`, dòng 93-97; `agents/README.md`, dòng 96-101)

8. Orchestration Patterns — Mẫu Điều phối

8.1. Năm Mẫu Được Chấp nhận

#	Pattern	Cost	Ví dụ
1	Direct invocation — 1 persona, 1 artifact	1 round trip	"Review PR này" → <code>code-reviewer</code>
2	Single-persona command — Wrap 1 persona + skill	= Direct	<code>/review</code> , <code>/test</code>
3	Parallel fan-out + merge — N personas song song	N contexts + 1 merge	<code>/ship</code>
4	Sequential pipeline — User là orchestrator	1 context/step	<code>/spec</code> → <code>/plan</code> → <code>/build</code> → <code>/test</code> → <code>/review</code> → <code>/ship</code>
5	Research isolation — Sub-agent đọc nhiều, trả digest nhỏ	1 isolated context	"Tìm mọi call site của deprecated API"
(Nguồn: <code>references/orchestration-patterns.md</code> , dòng 9-113)			

8.2. Bốn Anti-patterns Bị Cấm

- **A. Router persona** — Persona "meta-orchestrator" chỉ routing, không có domain value. Tốn 2× token.
- **B. Persona gọi persona** — Mất context, nhân failure modes, ẩn cost.
- **C. Sequential orchestrator tự paraphrase** — Mất human checkpoints, drift tích lũy.
- **D. Deep persona trees** — Nhiều tầng = latency + token vô ích. (Nguồn: `references/orchestration-patterns.md`, dòng 282-339)

8.3. Sub-agents vs Agent Teams

	Sub-agents (/ship)	Agent Teams (thử nghiệm)
Giao tiếp	Chỉ báo cáo ngược cho Agent chính	Teammates nhắn tin trực tiếp với nhau
Mục đích	Đưa phán quyết (verdict) trên artifact đã biết	Điều tra qua tranh luận đối kháng (competing hypotheses)
Khi dùng	PR review, pre-deploy check	Debug lỗi Production phức tạp
Trạng thái	Stable	Experimental (<code>CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1</code>)
(Nguồn: <code>references/orchestration-patterns.md</code> , dòng 125-278)		

9. Hooks System — Vòng đời Session

9.1. Session Start Hook

Mỗi khi Agent bắt đầu session mới, `hooks/session-start.sh` tự động inject nội dung meta-skill `using-agent-skills` vào context. Đảm bảo Agent luôn biết cách chọn đúng skill ngay từ đầu. (Nguồn: `hooks/hooks.json`; `hooks/session-start.sh`, dòng 1-25)

9.2. SDD-Cache (Source-Driven Development Cache)

Cache HTTP thông minh cho `source-driven-development`:

- Cache nội dung docs đã fetch lên đĩa (`.claude/sdd-cache/<sha>.json`).
- Luôn **revalidate** với server gốc qua `ETag / If-Modified-Since` .
- Chỉ serve cache khi server trả `304 Not Modified` — **không bao giờ trả docs cũ**.
- Server không có ETag/Last-Modified → **không cache**.

Yêu cầu: `jq` , `curl` , `shasum` , Bash 3.2+. (Nguồn: `hooks/SDD-CACHE.md` , dòng 1-168)

9.3. Simplify-Ignore Hook

Cho phép đánh dấu file/đoạn code không được phép simplify, bảo vệ code có chủ đích phức tạp. (Nguồn: `hooks/SIMPLIFY-IGNORE.md`)

10. Tương thích Đa nền tảng (Multi-platform)

10.1. Cài đặt với Cursor

`README.md` chỉ nêu ngắn gọn rằng có thể copy `SKILL.md` vào `.cursor/rules/` hoặc trỏ tới toàn bộ thư mục `skills/` (`README.md` , dòng 63-68). Tài liệu chi tiết hơn nằm ở `docs/cursor-setup.md` .

Cách cài khuyến nghị:

1. Tạo thư mục `.cursor/rules/` trong project.
2. Copy các skill quan trọng vào đây, ví dụ:
 - `test-driven-development`
 - `code-review-and-quality`
 - `incremental-implementation`
3. Để các skill dùng theo ngữ cảnh ở dạng Notepad thay vì load tất cả cùng lúc.

Ví dụ lệnh:

```
mkdir -p .cursor/rules
cp /path/to/agent-skills/skills/test-driven-development/SKILL.md .cursor/rules/test-dr
cp /path/to/agent-skills/skills/code-review-and-quality/SKILL.md .cursor/rules/code-re
cp /path/to/agent-skills/skills/incremental-implementation/SKILL.md .cursor/rules/incr
```

Lựa chọn khác:

- Dùng file `.cursorrules` để inline vài skill thiết yếu.
- Dùng Notepads của Cursor để lưu các skill theo từng phase và gọi lại bằng `@notepad` .

Lưu ý thực tế: không nên load toàn bộ skill cùng lúc vì Cursor có giới hạn context; cách hợp lý là giữ 2-3 skill lõi trong rules, còn lại chỉ nạp khi cần. (Nguồn: `README.md`, dòng 63-68; `docs/cursor-setup.md`)

10.2. Cài đặt với Gemini CLI

`README.md` cung cấp hướng dẫn cài khá rõ cho Gemini CLI (`README.md` , dòng 70-87). Ý chính là cài `agent-skills` như **native skills** để Gemini tự discover, hoặc thêm vào `GEMINI.md` nếu muốn giữ persistent context.

Cài trực tiếp từ repo GitHub:

```
gemini skills install https://github.com/addyosmani/agent-skills.git --path skills
```

Cài từ bản clone local:

```
gemini skills install ./agent-skills/skills/
```

Sau khi cài, Gemini CLI có thể tự nhận diện bộ skills và ánh xạ task vào skill phù hợp. Repo cũng có thư mục `.gemini/commands/` để cung cấp bộ command tương ứng cho môi trường Gemini. (Nguồn: `README.md`, dòng 70-87; thư mục `.gemini/commands/`)

10.3. Các nền tảng khác

Với các nền tảng còn lại, báo cáo này chỉ ghi đường dẫn tra cứu nhanh trong `README.md` :

- **Claude Code:** xem `README.md` , dòng 38-61.
- **Windsurf:** xem `README.md` , dòng 89-94.
- **OpenCode:** xem `README.md` , dòng 96-103.
- **GitHub Copilot:** xem `README.md` , dòng 105-110.
- **Kiro IDE & CLI:** xem `README.md` , dòng 112-115.
- **Codex / Other Agents:** xem `README.md` , dòng 117-122.

Bản chất chung: skills là các file Markdown, nên ngoài các tích hợp chính thức ở trên, bộ KIT vẫn dùng được với bất kỳ agent nào hỗ trợ system prompts hoặc rules files. (Nguồn: `README.md` , mục `Quick Start`; `docs/getting-started.md`)

11. Tổng kết Đánh giá

11.1. Ưu điểm (Pros)

- **Chất lượng Senior Engineer:** Code được bảo hành qua TDD, OWASP review, 5-axis code review.
- **Tính nhất quán:** AI không "ảo giác" nhờ workflow bắt buộc + anti-rationalization tables.
- **An toàn khi Deploy:** `/ship` fan-out 3 chuyên gia + bắt buộc Rollback Plan.
- **Tiết kiệm Token:** Progressive disclosure — skill chỉ load khi cần, SKILL.md dưới 500 dòng.
- **Đa nền tảng thực dụng:** Claude Code, Gemini CLI, Cursor, OpenCode, Windsurf, Copilot, Kiro đều được README hoặc docs nhắc cách dùng ở các mức độ khác nhau. (Nguồn: `README.md`; `docs/`)

11.2. Nhược điểm (Cons)

- **Chi phí context/token cao hơn cho task lớn:** Đây là nhận định phân tích. Chu trình `/spec` → `/plan` → `/build` và fan-out `/ship` tạo thêm artifact, thêm lượt suy luận, thêm context window so với việc prompt trực tiếp.
- **Có độ cứng quy trình với task nhỏ:** Đây cũng là nhận định phân tích. Với các thay đổi rất nhỏ, việc luôn nghĩ theo spec, task breakdown, verification và review có thể bị cảm nhận là nặng tay, dù `/ship` cho phép bỏ qua fan-out với diff nhỏ.
- **Hooks cần Bash + jq:** SDD-Cache và session-start yêu cầu Bash 3.2+, `jq`, `curl` — không thân thiện trên Windows thuần. (Nguồn: `.claude/commands/ship.md`, phần `rules`; `hooks/SDD-CACHE.md`, phần `requirements`; hai ý đầu là phân tích tổng hợp của người viết)

11.3. Kịch bản Áp dụng Tối ưu (Use Cases)

1. **Dự án Mới / Module phức tạp:** `/spec` → `/plan` → `/build` để AI hiểu toàn cảnh trước khi code.
2. **Legacy Code Refactoring:** `/code-simplify` với Chesterton's Fence + test liên tục sau mỗi thay đổi.
3. **Debug lỗi Production:** `/test` với Prove-It pattern. Bug cần competing hypotheses → Agent Teams.
4. **Pre-deploy tính năng nhạy cảm:** `/ship` để 3 sub-agents quét OWASP, coverage gaps, architectural issues.
5. **Migration hệ thống cũ:** Skill `deprecation-and-migration` + `/code-simplify` để dọn zombie code. (Nguồn: Phân tích tổng hợp từ toàn bộ KIT)

12. Demo Full Workflow với PRD TaskFlow

12.1. Mục tiêu Demo

Demo này dùng trực tiếp `note/PRD.md` của sản phẩm **TaskFlow** để cho team thấy cách `agent-skills` biến một PRD khá đầy đủ thành quy trình triển khai có kiểm soát, có bằng chứng xác minh và có điểm chặn chất lượng trước khi ship.

Phạm vi demo được sửa lại để bao phủ **cả 12 Functional Requirements (FR-01 → FR-12)** và **toàn bộ User Stories (US-01 → US-05)** trong PRD, nhưng **không làm một cục**. Cách chạy đúng theo KIT là:

`/spec` một lần để chốt yêu cầu, `/plan` một lần để chia slice, sau đó xử lý tuần tự từng slice. Mỗi slice phải chạy `/build` → `/test` → `/review`; nếu chưa pass thì quay lại sửa ngay trong slice đó, không được chuyển sang slice tiếp theo. Chỉ khi tất cả slice pass mới chạy `/ship`. (Nguồn: `note/PRD.md`, toàn văn; `README.md`, phần *Commands*)

12.2. Input từ PRD và các giả định chốt cho demo

Persona dùng trong demo:

- **Minh** — Team Lead / Manager: cần tạo task, giao việc, theo dõi tiến độ.
- **Linh** — Member: cần thấy việc của mình và cập nhật trạng thái mà không phải báo qua chat.
- **Admin workspace**: cần quản lý workspace, thành viên, role và invite flow.

Phần PRD được chọn làm source truth:

- **Functional Requirements**: FR-01 đến FR-12, bao gồm authentication, workspace/member, project, task, status, comment, My Tasks, Team Dashboard, notification, activity log, reports và global search.
- **User Stories**: US-01 đến US-05, bao gồm tạo task, cập nhật trạng thái, xem My Tasks, invite member và xem báo cáo team.
- **NFR và edge cases**: dùng làm quality gates trong `/test`, `/review`, `/ship`, đặc biệt là security, usability, accessibility, data isolation và performance.

Giả định để tránh vướng open questions:

- Mỗi task chỉ có **1 assignee**.
- Notification trong demo chỉ là **in-app**, không mở rộng sang email.
- Không demo sub-task, SSO, mobile native app hay tích hợp Slack/Zalo. (Nguồn: `note/PRD.md`, mục 4, 7, 9, 17)

12.3. Phạm vi 12 FR và các US

Demo được chia thành các slice độc lập đủ nhỏ để trình bày từng phần. Mỗi slice vẫn là **vertical slice** đúng nghĩa: có backend, UI, test và verification đủ để chứng minh hành vi end-to-end, không chia kiểu “làm hết backend trước rồi mới làm frontend”.

Nhóm demo	FR liên quan	US liên quan	Kết quả cần chứng minh
Identity & workspace	FR-01, FR-02	US-04	User đăng ký/đăng nhập, Admin invite member, role được áp dụng đúng.
Project & task core	FR-03, FR-04	US-01	Manager tạo project/task, validate title, assign member, task xuất hiện đúng project.

Nhóm demo	FR liên quan	US liên quan	Kết quả cần chứng minh
Workflow execution	FR-05, FR-10	US-02	Assignee/Manager đổi status, activity log ghi đầy đủ, user sai quyền bị chặn.
Collaboration	FR-06, FR-09	US-01, US-02	Comment, mention và notification in-app hoạt động đúng rule.
Personal productivity	FR-07	US-03	<code>My Tasks</code> hiển thị đúng task assigned, sort overdue/duel date đúng, empty state đúng.
Team visibility	FR-08	US-02	Manager xem Kanban board, filter/search trong board, status cập nhật trong thời gian kỳ vọng.
Reporting	FR-11	US-05	Reports hiển thị completed theo tuần, completion rate và overdue theo member.
Discovery	FR-12	US-01, US-03, US-05	Global search tìm task theo title, debounce và giới hạn kết quả đúng PRD.

Điểm khác biệt so với bản demo cũ: bản cũ chỉ chứng minh FR-04/05/07/08 và US-01/02/03; bản sửa này dùng KIT để đi hết scope MVP FR-01 → FR-12, đồng thời nối từng FR với user story hoặc quality gate cụ thể. (Nguồn: `note/PRD.md`, mục 7 và mục 9)

12.4. Full Workflow theo `agent-skills`

Nguyên tắc quan trọng: không chạy `/build` một lần cho toàn bộ PRD. KIT phải ép agent đi theo vòng nhỏ:

```
/spec → tạo SPEC.md cho toàn PRD
/plan → chia SPEC thành Slice A-H
```

```
Slice A: /build → /test → /review → pass? nếu không pass, quay lại /build Slice A
Slice B: /build → /test → /review → pass? nếu không pass, quay lại /build Slice B
Slice C: /build → /test → /review → pass? nếu không pass, quay lại /build Slice C
...
Slice H: /build → /test → /review → pass? nếu không pass, quay lại /build Slice H
```

```
/ship → chỉ chạy sau khi mọi slice đã pass gate
```

Như vậy, báo cáo demo không trình bày “làm 12 FR một lượt”, mà trình bày cách KIT kiểm soát tiến độ theo từng lát nhỏ có evidence. Slice sau không được bắt đầu nếu slice trước còn fail test, còn finding blocker, hoặc chưa có bằng chứng acceptance criteria.

Bước 1 — /spec : Chuyển PRD thành spec làm việc

Agent nhận `note/PRD.md` làm source truth, không viết spec từ khoảng trống. Ở bước này, agent trích ra:

- **Objective:** thay thế chat/spreadsheet bằng một nơi duy nhất để assign, track và report công việc.
- **Target users:** Admin, Manager và Member trong team nhỏ 5-10 người.
- **Functional scope:** toàn bộ FR-01 → FR-12, chia theo phase Identity, Core Task, Collaboration, Dashboards, Reports, Search.
- **Acceptance criteria:** lấy trực tiếp từ US-01 → US-05.
- **Boundaries:** không mở rộng sang email notification ngoài invite, sub-task, SSO, native mobile, Slack/Zalo, billing hoặc multiple workspaces.

Artifact kỳ vọng: `SPEC.md` có requirement matrix: mỗi FR có owner role, endpoint/UI dự kiến, acceptance evidence và test coverage tối thiểu. Phần testing strategy phải bám sát PRD thay vì tự phát minh thêm yêu cầu.

Giá trị của skill: `spec-driven-development` buộc agent phải chốt rõ "xây gì" và "không xây gì" trước khi bước vào code.

Bước 2 — /plan : Chia công việc thành vertical slices

Từ `SPEC.md`, agent dùng `planning-and-task-breakdown` để tách công việc thành task nhỏ, có thứ tự phụ thuộc:

- **Slice A — Foundation:** FR-01, FR-02. Auth, workspace, member invite, role model.
- **Slice B — Project/Task CRUD:** FR-03, FR-04. Project, task form, validation, assign.
- **Slice C — Status & Audit:** FR-05, FR-10. Status transition, authorization, activity log.
- **Slice D — Collaboration:** FR-06, FR-09. Comment, mention, notification in-app.
- **Slice E — Personal Dashboard:** FR-07. My Tasks, sort, overdue, empty state.
- **Slice F — Team Dashboard:** FR-08. Kanban, filters, search trong board, manager visibility.
- **Slice G — Reports:** FR-11. Completed chart, member table, overdue.
- **Slice H — Global Search & Polish:** FR-12 + NFR. Header search, debounce, performance, accessibility, security hardening.

Mỗi slice phải có acceptance criteria và verification step đi kèm, ví dụ:

- đăng nhập hết hạn thì redirect về login,
- invite link hết hạn sau 48 giờ,
- tạo task thành công với `Title` + `Project`,
- hiện lỗi nếu `Title` trống,
- assignee thấy task mới trong `My Tasks`,

- Manager thấy status đổi trong board,
- Reports chỉ tính dữ liệu trong workspace hiện tại,
- global search trả tối đa 10 kết quả.

Artifact kỳ vọng: `tasks/plan.md` và `tasks/todo.md`.

Giá trị của skill: tránh kiểu chia ngang theo “frontend trước, backend sau”; mỗi slice đều có giá trị sản phẩm kiểm chứng được và vẫn tiến dần tới đủ 12 FR.

Bước 3 — `/build`: Chỉ build một slice tại một thời điểm

Agent không build toàn bộ một lần. Mỗi lần `/build` chỉ nhận **một slice** từ `tasks/todo.md`. Với slice đó, KIT yêu cầu đi theo vòng lặp nhỏ và đóng gate trước khi qua slice sau:

1. Đọc FR/US tương ứng trong `note/PRD.md`.
2. Viết test thất bại hoặc checklist verification trước.
3. Implement mức tối thiểu để test pass.
4. Chạy `/test` cho slice.
5. Chạy `/review` cho slice.
6. Nếu test fail hoặc review có blocker, quay lại `/build` để sửa đúng slice đó.
7. Chỉ chuyển sang slice tiếp theo khi slice hiện tại đạt acceptance gate.

Ví dụ cách trình bày trong demo:

- **Lượt 1:** `/build Slice A` cho `auth/workspace` → `/test Slice A` → `/review Slice A` → nếu pass mới qua Slice B.
- **Lượt 2:** `/build Slice B` cho `project/task CRUD` → `/test Slice B` → `/review Slice B` → nếu pass mới qua Slice C.
- **Lượt 3:** `/build Slice C` cho `status/activity log` → `/test Slice C` → `/review Slice C` → nếu pass mới qua Slice D.
- **Các lượt sau:** tiếp tục Slice D-H theo cùng nhịp; không nhảy cóc sang toàn bộ hệ thống, không gom review đến cuối.

Dependency order hợp lý:

- **A trước B:** chưa có `auth/workspace` thì chưa thể đảm bảo task thuộc workspace nào.
- **B trước C/E/F:** chưa có task thì chưa thể đổi status, xem My Tasks hoặc Kanban.
- **C trước Reports:** `activity/status` là nguồn dữ liệu cho completion và overdue.
- **D sau core task:** comment/notification cần task và assignee đã ổn định.
- **H cuối:** search, performance, accessibility và hardening cần dữ liệu/UI đủ rộng để kiểm.

Ở bước này, các skill phụ có thể cùng tham gia:

- `incremental-implementation` để cắt lát dọc, không làm quá scope,
- `test-driven-development` để buộc có bằng chứng,
- `api-and-interface-design` để chốt contract cho auth, task, reports và search,
- `frontend-ui-engineering` để đảm bảo form, dashboard, Kanban và report usable/accessibility đúng NFR,
- `security-and-hardening` khi xử lý auth, role, workspace isolation và input sanitization.

Artifact kỳ vọng: sau mỗi slice có code thay đổi nhỏ, test pass theo FR/US liên quan, review không còn blocker và evidence rõ ràng trước khi tiếp tục. Không có artifact nào yêu cầu “xong hết 12 FR rồi mới kiểm”.

Bước 4 — `/test` : Test theo từng slice, fail thì không được qua slice sau

Đây là bước biến “chạy được” thành “chứng minh được”. `/test` phải chạy ngay sau từng slice, không chờ đến cuối. Nếu fail, agent phải quay lại `/build` cùng slice để sửa, rồi chạy lại `/test`.

Test gate theo slice:

- **FR-01:** đăng ký/đăng nhập bằng email/password, token hết hạn redirect login, logout kết thúc session.
- **FR-02 + US-04:** Admin invite member, link hết hạn sau 48 giờ, email đã là member thì không gửi invite, role thay đổi có hiệu lực.
- **FR-03:** Manager tạo/archive project, task count và done count hiển thị đúng.
- **US-01:** Manager tạo task với `Title` + `Project` thì task được tạo ở trạng thái `To Do`, ghi activity log, assignee nhận notification in-app. Nếu `Title` trống thì trả lỗi và không tạo task.
- **US-02:** Assignee đổi status sang `In Progress` thì UI cập nhật ngay, activity log có bản ghi, `Team Dashboard` phản ánh thay đổi. Nếu không phải assignee/manager thì nút đổi status bị disable.
- **US-03:** `My Tasks` hiển thị task assigned cho user, sort overdue trước, due date tăng dần, task không có due date xuống cuối, empty state đúng khi không có dữ liệu.
- **FR-06:** member comment được trong task, mention `@username` tạo notification đúng người.
- **FR-08:** Manager xem Kanban, filter theo assignee/project/priority/due date và search theo title.
- **FR-09:** notification badge/dropdown/read state hoạt động cho assign, comment, due soon và mention.
- **FR-10:** activity log ghi create/edit/status/comment, không cho xóa log.
- **FR-11 + US-05:** Reports hiển thị bar chart 4 tuần, bảng Member/Assigned/Completed/Overdue/Completion Rate, dữ liệu chỉ nằm trong workspace hiện tại.
- **FR-12:** global search debounce 300ms, trả tối đa 10 task theo title trong workspace.
- **NFR:** page/API đạt ngưỡng performance, form có label, Kanban có keyboard fallback, input được sanitize, dữ liệu workspace không bị lộ chéo.

Artifact kỳ vọng: mỗi slice có kết quả test hoặc checklist xác minh riêng. Kết quả fail là gate đóng, không phải ghi chú để xử lý sau. Sau Slice H mới chạy regression toàn bộ FR-01 → FR-12 và US-01 → US-05.

Bước 5 — `/review` : Review nhỏ sau từng slice, blocker thì sửa ngay

Tại bước này, `code-review-and-quality` được dùng sau mỗi slice để tránh tích lũy lỗi lớn. Nếu review phát hiện blocker, agent quay lại `/build` đúng slice đó rồi chạy lại `/test` và `/review`. Review rà theo 5 trục:

- **Correctness:** hành vi có khớp FR-01 → FR-12 và US-01 → US-05 không.
- **Readability:** tên entity, status, role, filter, report metric và search behavior có rõ ràng không.
- **Architecture:** boundaries giữa auth, workspace isolation, task service, notification, activity log, dashboard và reports có sạch không.
- **Security:** role Admin/Manager/Member được enforce server-side; workspace isolation không phụ thuộc UI; input task/comment/search được validate/sanitize.
- **Performance:** query My Tasks, Kanban, Reports và Global Search có nguy cơ N+1, full scan hoặc reload dư thừa không.

Ngoài ra, demo này nên gọi rõ thêm hai điểm review quan trọng từ PRD:

- **Accessibility:** form có label rõ ràng, Kanban có keyboard fallback.
- **Usability:** tạo task và đổi status không vượt quá 3 click.

Artifact kỳ vọng: danh sách findings có severity theo từng slice. Finding blocker của slice nào phải sửa ngay ở slice đó, không đẩy sang cuối.

Bước 6 — `/ship` : Quyết định GO/NO-GO cho staging

Khi toàn bộ 12 FR và 5 US đã qua test/review, `/ship` đóng vai trò pre-launch orchestrator:

- `code-reviewer` kiểm correctness/readability/architecture/performance,
- `security-auditor` rà authz, validation, secrets/input handling,
- `test-engineer` rà coverage gaps cho happy path, edge case, error path.

Main agent sau đó hợp nhất kết quả và đưa ra:

- **GO** nếu không còn blocker,
- hoặc **NO-GO** nếu còn lỗi về auth/session, invite, workspace isolation, phân quyền status, thiếu activity log, notification sai người, report sai số, search lộ dữ liệu workspace khác, hoặc dashboard/My Tasks sort sai so với PRD.

Rollback plan tối thiểu cho demo staging:

- **Trigger conditions:** login lỗi, invite lỗi, tạo task lỗi, đổi status sai quyền, dashboard/report/search không đúng dữ liệu, notification leak sang user khác.
- **Rollback procedure:** tắt feature flag cho module lỗi hoặc quay về bản staging trước đó.
- **Recovery goal:** khôi phục workflow TaskFlow ổn định ở mức tối thiểu: auth, task CRUD, status update, My Tasks và Team Dashboard.

12.5. Kịch bản Trình bày cho Toàn Team

1. Mở đầu bằng problem statement của TaskFlow: team nhỏ đang giao việc qua chat và spreadsheet nên thiếu visibility, accountability, data.
2. Chỉ ra Admin, Minh và Linh để cả team hiểu demo đi qua đủ role chính trong PRD.
3. Mở `PRD.md`, highlight FR-01 → FR-12 và US-01 → US-05 để chốt phạm vi đầy đủ.
4. Trình bày `/spec`: từ PRD rút ra objective, users, requirement matrix, acceptance criteria và boundaries.
5. Trình bày `/plan`: chia thành 8 slices A → H, giải thích dependency order và vì sao vẫn là vertical slice chứ không chia ngang frontend/backend.
6. Trình bày `/build` + `/test`: cho thấy agent build từng slice theo RED → GREEN và phải có evidence cho từng FR/US.
7. Kết thúc bằng `/review` + `/ship`: nhấn mạnh điểm khác biệt của KIT là có quality gate, security audit, test coverage review và rollback thinking trước khi kết luận GO/NO-GO.

12.6. Prompt mẫu cho từng bước

Các prompt dưới đây giúp buổi demo không bị nói chung chung. Người demo có thể copy từng prompt vào agent theo đúng thứ tự. Nguyên tắc là prompt luôn chỉ rõ source truth, scope, artifact kỳ vọng và gate pass/fail.

Prompt 1 — `/spec`: tạo SPEC từ PRD

`/spec`

Đọc note/PRD.md làm source truth duy nhất cho TaskFlow.

Tạo SPEC.md cho MVP, bao phủ FR-01 đến FR-12 và US-01 đến US-05.

Yêu cầu SPEC.md phải có:

- Objective và problem statement lấy từ PRD.
- Roles: Admin, Manager, Member.
- Requirement matrix: FR | user value | UI/API surface | acceptance evidence | test ty
- Boundaries: không làm SSO, mobile native, Slack/Zalo, billing, multiple workspaces,
- Open assumptions: mỗi task chỉ có 1 assignee, notification demo là in-app.

Không viết code ở bước này.

Nếu PRD mâu thuẫn hoặc thiếu thông tin blocking, dừng và liệt kê câu hỏi.

Prompt 2 — /plan : chia SPEC thành slice

/plan

Dựa trên SPEC.md và note/PRD.md, tạo tasks/plan.md và tasks/todo.md.

Chia implementation thành các vertical slices tuần tự:

- A. Foundation: FR-01, FR-02, US-04.
- B. Project/Task CRUD: FR-03, FR-04, US-01.
- C. Status & Audit: FR-05, FR-10, US-02.
- D. Collaboration: FR-06, FR-09.
- E. Personal Dashboard: FR-07, US-03.
- F. Team Dashboard: FR-08.
- G. Reports: FR-11, US-05.
- H. Global Search & Polish: FR-12 + NFR.

Với mỗi slice, ghi rõ:

- Scope in/out.
- Files/modules dự kiến chạm vào.
- Acceptance criteria lấy từ PRD.
- Test gate phải pass.
- Review gate phải pass.

Không build code ở bước này.

Prompt 3 — /build cho từng slice

/build Slice A

Chỉ implement Slice A trong tasks/todo.md: Foundation gồm FR-01, FR-02 và US-04.

Quy tắc:

- Không làm sang Slice B-H.
- Viết hoặc cập nhật test/checklist trước khi implement.
- Implement tối thiểu để pass acceptance criteria của Slice A.
- Sau khi build xong, liệt kê file đã đổi và bằng chứng cần chạy ở /test Slice A.

Khi qua slice tiếp theo, chỉ thay tên slice và scope:

`/build Slice B`

Chỉ implement Slice B trong `tasks/todo.md`: Project/Task CRUD gồm FR-03, FR-04 và US-01. Không làm sang Slice C-H.

Viết hoặc cập nhật `test/checklist` trước khi implement.

Implement tối thiểu để pass acceptance criteria của Slice B.

Sau khi build xong, liệt kê file đã đổi và bằng chứng cần chạy ở `/test Slice B`.

Prompt 4 — `/test` cho từng slice

`/test Slice A`

Chạy `test/verification` cho Slice A theo `tasks/todo.md` và `note/PRD.md`.

Gate phải kiểm:

- FR-01: đăng ký, đăng nhập, token hết hạn redirect login, logout.
- FR-02/US-04: invite member, link 48 giờ, email đã là member thì báo lỗi, role có hiệu lực.

Output bắt buộc:

- PASS/FAIL.
- Lệnh test hoặc checklist đã chạy.
- Evidence ngắn cho từng acceptance criteria.
- Nếu FAIL, chỉ rõ lỗi blocking và không chuyển sang Slice B.

Prompt 5 — `/review` cho từng slice

`/review Slice A`

Review phần thay đổi của Slice A sau khi `/test Slice A` đã chạy.

Tập trung vào:

- Correctness so với FR-01, FR-02, US-04.
- Security: password hashing, session/token handling, role enforcement, workspace isolation.
- Architecture: auth/workspace/member boundary rõ ràng.
- Error states: invite expired, duplicate member, invalid session.
- Missing tests hoặc acceptance criteria chưa có evidence.

Output:

- PASS nếu không có blocker.
- NO-GO nếu còn blocker, ghi rõ phải quay lại `/build Slice A` để sửa.

Prompt 6 — sửa khi slice fail

`/build Slice A – fix review/test blockers`

Chỉ sửa các blocker được nêu trong `/test Slice A` hoặc `/review Slice A`.

Không mở rộng scope, không làm sang slice khác.

Sau khi sửa xong, yêu cầu chạy lại `/test Slice A` và `/review Slice A`.

Prompt 7 — regression sau Slice H

`/test regression`

Tất cả Slice A-H đã pass riêng lẻ.

Chạy regression toàn bộ MVP theo note/PRD.md:

- FR-01 đến FR-12.
- US-01 đến US-05.
- NFR quan trọng: security, workspace isolation, performance, accessibility, usability

Output PASS/FAIL, evidence tổng hợp và danh sách blocker nếu có.

Nếu regression FAIL, chỉ rõ slice nào phải quay lại sửa.

Prompt 8 — `/ship`: quyết định GO/NO-GO

`/ship`

Chỉ chạy sau khi Slice A-H đã pass `/build`, `/test`, `/review` và regression đã pass.

Hãy tổng hợp release decision cho staging:

- Scope đã hoàn thành theo FR-01 đến FR-12 và US-01 đến US-05.
- Test evidence chính.
- Review/security/test coverage findings còn lại.
- GO/NO-GO.
- Rollback plan: trigger conditions, rollback procedure, recovery goal.

Nếu còn blocker về auth, workspace isolation, permission, activity log, notification,

Cách dùng trong demo: chạy prompt 1 và 2 để tạo artifact nền. Sau đó lặp Prompt 3 → 4 → 5 cho từng slice. Nếu fail, dùng Prompt 6 cho đến khi pass. Sau Slice H, dùng Prompt 7 rồi mới dùng Prompt 8.

12.7. Giá trị Rút ra từ Demo

- **Với team sản phẩm:** PRD không bị “đọc rồi để đó”; nó trở thành source truth xuyên suốt từ spec đến ship.

- **Với team kỹ thuật:** `agent-skills` giảm rủi ro AI code quá nhanh nhưng thiếu test, thiếu review, thiếu security check.
- **Với buổi trình bày nội bộ:** demo không chỉ chứng minh một luồng task nhỏ, mà cho thấy KIT có thể điều phối toàn bộ MVP gồm 12 FR và 5 US từ requirement đến verification và launch decision.

12.8. Kết luận cho phần Demo

Nói ngắn gọn: **Agent Skills KIT không chỉ giúp “làm ra tính năng”, mà giúp biến toàn bộ PRD thành một workflow phát triển có kiểm soát, có bằng chứng và có cơ chế chặn lỗi trước khi ship.**